

Brick and the Mixture-of-Models (MoM) Paradigm

Bridging Open- and Closed-Weight LLM Pools

 **Seeweb**  **Regolo.ai** • **May 2026**

Francesco Massa
f.massa@regolo.ai

Marco Cristofanilli
marco.c@seeweb.it

 [regolo-ai/brick-SR1](https://github.com/regolo-ai/brick-SR1) •  huggingface.co/regolo

Abstract. What is difficult for a language model? Defining difficulty is one of the hardest problems in deployment engineering, and the only reliable answer is empirical. Once correctness is measured per query and per model, the question becomes: can a router dispatch each query to the cheapest model that will answer it correctly?

Existing systems route on surface features: domain labels, keywords, token count. We call this *superficial routing*. It collapses the within-domain variance that actually determines which model succeeds, since a one-line question can target an unsolved problem while a long boilerplate prompt can wrap a trivial task.

This paper takes a further step toward the *Mixture of Models* (MoM) paradigm, where an external router dispatches each query to the cheapest full model in a heterogeneous pool. We present **Brick**, a multimodal router that scores each model on six capability dimensions and combines this with a per-query difficulty estimate to produce a cost-penalized score. A continuous preference knob r lets operators slide between a *max-quality* profile and a *max-saving* profile, turning the quality-versus-spend trade-off into a deploy-time parameter rather than a code change. On **Dataset A** (5,504 queries, $\kappa=0.761$), Brick at max-quality reaches **76.98%** accuracy, beating the best single model (kimi2.6, 75.02%) and all external routers (RouteLLM, FrugalGPT, Cascade Routing); at max-saving it cuts average cost per query by $4\times$ against always-kimi while staying within 3 pp of its accuracy. At production scale, even small per-request savings compound across millions of calls, making the knob a direct cloud-bill lever. Median end-to-end latency is 22.8 s vs 51.2 s for always-kimi; the three-model oracle bound is 83.25%.

Keywords. LLM routing, mixture of models, capability classification, cost-aware inference, open-weight models.

1. INTRODUCTION

Identifying what is *difficult* for a given language model is one of the harder skills a deployment engineer can develop. Training data, architecture changes, and reinforcement post-training all interact in opaque ways, so deciding which model is best for a task is in practice an empirical exercise: only repeated evaluation reveals where a model excels and where it fails silently.

A growing collection of public evaluation suites makes this exercise tractable. Once response correctness can be measured per query and per model, the question becomes: given a pool of K candidate models with heterogeneous price and capability profiles, can a router send each query to the cheapest model that will answer it correctly? This is the routing problem we address. Two engineering constraints make it non-trivial. First, capability is multi-dimensional; a single model can be strong at code while weak at planning, so global ranking is misleading. Second, both cost and latency vary across models by an order of magnitude, so the wrong choice penalizes either accuracy or the

bill.

This paper contributes:

1. The *Mixture-of-Models* (MoM) paradigm as a deployment framing for routing across heterogeneous full-weight models including mixed open-weight and commercial pools (§2).
2. A critique of two superficial routing strategies that fail to capture intra-domain difficulty (§3).
3. **Brick**, a calibrated MoM router with an explicit capability vector representation and a user-facing preference knob (§8).
4. Experimental results on Dataset A showing Brick exceeds the best single-model baseline at a fraction of its perceived latency (§9, §10).

2. THE MIXTURE-OF-MODELS (MOM) PARADIGM

Training a new frontier model from scratch costs tens of millions of dollars and months of compute. A simpler question is: why not use the models already available? The market offers a wide range of pretrained

models at different price points and capability profiles. Rather than collapsing this diversity into a single choice, we decided to build a custom architecture on top of it. We call this the *Mixture of Models* (MoM) paradigm: a pool of whole pretrained models, dispatched one per query through an external router. The unit of routing is a model, not a layer. MoM differs from *Mixture of Experts* (MoE), where sparse sub-networks inside a single model are activated per token; here the mixing happens at deployment time, across independent systems.

MoM is appealing for three operational reasons:

- **Heterogeneous pricing.** Frontier commercial models charge ten to one hundred times what local open-weight models cost. A router that uses them only when needed makes the median query cheap without losing the long tail.
- **OW+CW bridging.** An MoM pool can mix open-weight models hosted in a controlled environment with closed-weight commercial APIs. The router lets an operator capture the data-locality and price benefits of open weights on easy queries while reserving expensive closed-weight calls for hard ones.
- **Capability complementarity.** Models from different vendors and training recipes often have complementary error sets. As shown in §5, the three-model oracle on Dataset A reaches 83.25%, well above any single model.

The rest of this paper takes Brick as a concrete instantiation of MoM. The geometric routing rule and the calibration procedure are properties of Brick, not of the MoM paradigm itself; alternative MoM realizations are possible and are discussed in §11.

3. LIMITATIONS OF SUPERFICIAL ROUTING

Two routing strategies recur in production systems and deserve to be ruled out before introducing Brick.

Domain-based routing maps the query to a coarse domain (science, math, humanities, code) and dispatches to a per-domain best model. The hidden assumption is that, conditional on the domain, models rank uniformly. This assumption fails because difficulty varies sharply inside a domain: a request to write a Python calculator and a request to solve a Riemann-zeta question are both “code” or both “math” depending on framing, but require different models. Domain-based routing collapses this variance and over-pays on easy queries.

Length- and keyword-based routing maps the query to a complexity score from token count and regex hits. The assumption is that long or jargon-rich queries are hard, and short queries are easy. This also fails: a one-line question may target an unsolved problem, while a long boilerplate prompt may wrap a trivial transformation. Token count is a poor proxy for required

capability.

The remainder of this paper treats routing as a decision in capability space, not in surface-feature space.

4. MODEL POOL

We use a fixed three-model pool:

$$\mathcal{M} = \{\text{qwen3.5-9b}, \text{deepseek-v4-flash}, \text{kim2.6}\}.$$

The pool spans three quality-and-cost regimes: qwen3.5-9b as a state-of-the-art under-10B-parameter open-weight model, deepseek-v4-flash as a state-of-the-art under-500B open-weight model, and kim2.6 as a frontier open-source contender. Per-call normalized costs used in the routing math are

$$a_{\text{qwen}}=0.10, \quad a_{\text{ds4}}=0.40, \quad a_{\text{kim2.6}}=0.60.$$

These dimensionless ratios are calibrated to public per-token pricing at the time of evaluation. They preserve the relative cost gap rather than committing to absolute USD values that drift between vendors. Formally, given the public output price p_m in USD per 10^6 tokens, the cost scalar is

$$a_m = \frac{p_m}{p_{\text{ref}}}, \quad p_{\text{ref}} = 1.00 \text{ USD}/10^6 \text{ tokens}.$$

At evaluation time (May 2026) the reference prices were $p_{\text{qwen}}=0.10$, $p_{\text{ds4}}=0.40$, $p_{\text{kim2.6}}=0.60$ USD per 10^6 output tokens, yielding the values above directly. The scalar a_m is locked into the Brick configuration at calibration time and is not updated at inference.

5. DATASET A

Brick is evaluated on Dataset A, a stratified routing-evaluation benchmark of $N=5,504$ queries covering six capability dimensions. Table 1 summarizes the per-source composition. The corpus is built from fourteen license-clean upstream sources and three custom curated subsets. Each row carries a deterministic identifier, a typed evaluation payload, pre-computed token counts under the three target tokenizers, and a license tag enabling compliant redistribution.

5.1. Quality validation and oracle bound

The construction pipeline is validated in three tiers: deterministic schema and length checks, LLM-as-judge calibration against an audited reference subset, and a dual-reviewer manual audit on $n=102$ stratified samples. The manual audit yields Cohen’s $\kappa=0.761$ on a 4-class rubric and $\kappa=0.951$ on the binary collapse, both above the conventional substantial-agreement threshold.

Aggregate per-model response accuracy on Dataset A is reported in Table 2, together with the achievable three-model oracle bound. A query is counted as solved if the model’s response passes the per-protocol grader. The oracle counts a query as solved if any of

Table 1. Per-source composition of Dataset A ($N=5,504$). Counts may differ from the original release ($N=5,339$) due to the addition of 165 planning multi-turn rows in revision v0.4.

Dimension	Source	Count	License
instruction_following	IFEval [4]	541	apache-2.0
	IFBench [5]	300	ODC-BY-1.0
coding	LiveCodeBench-v6 [7]	1,000	cc-by-4.0
math_reasoning	MATH-500 [1]	500	mit
	AIME-2025	30	qwen-derived
	GSM8K [2]	470	mit
world_knowledge	SimpleQA [8]	700	mit
	MMLU-Pro-Humanities [3]	102	mit
	GPQA-Diamond [9]	0 (gated)	cc-by-4.0
creative_synthesis	EQ-Bench-Creative-v3 [10]	96	EQ-Bench terms
	LitBench [11]	500	qwen-derived
	Custom-Validated	100	qwen-derived
planning_agentic	BFCL-v4 [6]	500	apache-2.0
	tau-bench [12]	165	mit
	Planning-Custom (incl. multi-turn)	500	qwen-derived
Total		5,504	

the three models would have solved it; the residual 922 queries are unsolvable by the current pool.

Table 2. Standalone per-model response accuracy and three-model oracle bound on Dataset A.

System	Solved	Accuracy
always  qwen	3,477/5,504	63.17%
always  ds4	4,056/5,504	73.69%
always  kimi	4,129/5,504	75.02%
three-model oracle	4,582/5,504	83.25%
unsolvable (any model)	922/5,504	16.75%

Notably, *kimi2.6* is not the per-query oracle: it fails 1,375 queries, of which 453 are correctly answered by at least one cheaper model. An oracle restricted to *kimi+ds4* reaches 81.96%, a gain of 6.94 pp over *kimi* alone. This residual headroom is the upside an MoM router can in principle capture.

The dataset is publicly available at [regolo/brick-dataset-A-routing-eval](https://regolo.github.io/brick-dataset-A-routing-eval/).

6. EXTERNAL ROUTER BASELINES

We benchmark four published routing strategies on Dataset A under the same three-model pool: RouteLLM in its binary and tournament variants [13], FrugalGPT-style cascade routing [14], and Cascade Routing [15]. We use the public reference checkpoints in zero-shot mode; no fitting is performed on Dataset A.

Three observations are worth recording. First, RouteLLM, trained on a binary preference signal be-

tween two models, collapses on the three-model pool: it dispatches 99.96% of queries to *kimi*, behaving as a noisy always-*kimi*. Second, FrugalGPT in cascade mode terminates at the cheapest model whenever its self-verification gates fire; in a three-model pool with cost ratio 1:4:6, this collapses to always-*qwen* accuracy. Third, Cascade Routing routes to *ds4* on the majority of queries but does not match a tuned MoM router on aggregate accuracy.

The wider methodological observation is that cascade-style approaches commit to the cheapest model first and pay duplicate inference cost on rejection, which inflates perceived latency. Binary routers do not extend cleanly to $K \geq 3$ pools without recursive pairing. Both classes are misaligned with MoM, which selects a single model per query in one decision step.

7. COST VS PERFORMANCE FRONTIER

Figure 1 plots response accuracy against average normalized cost for the three single-model baselines, the four external routers, the five Brick preference profiles introduced in §8, and the three-model oracle ceiling. Brick’s max-quality profile dominates always-*kimi* on both axes: higher accuracy at lower cost. The Brick curve traces a Pareto front above the external routers across the operating range.

Table 3. External router baselines on Dataset A. Accuracy is response accuracy under the per-protocol grader. Distribution reports selected-model shares as percentages (qwen / ds4 / kimi).

Router	Accuracy	Avg. cost	Distribution q / ds4 / kimi (%)
RouteLLM binary	21.31%	0.999	0.04/0/99.96
RouteLLM tournament	21.31%	0.999	0.04/0/99.96
FrugalGPT cascade	63.17%	0.070	100/0/0
Cascade Routing	28.96%	0.526	18/61/21
always  qwen	63.17%	0.100	100/0/0
always  ds4	15.55%	0.500	0/100/0
always  kimi	21.28%	1.000	0/0/100
oracle	100.00%	0.335	63/16/21

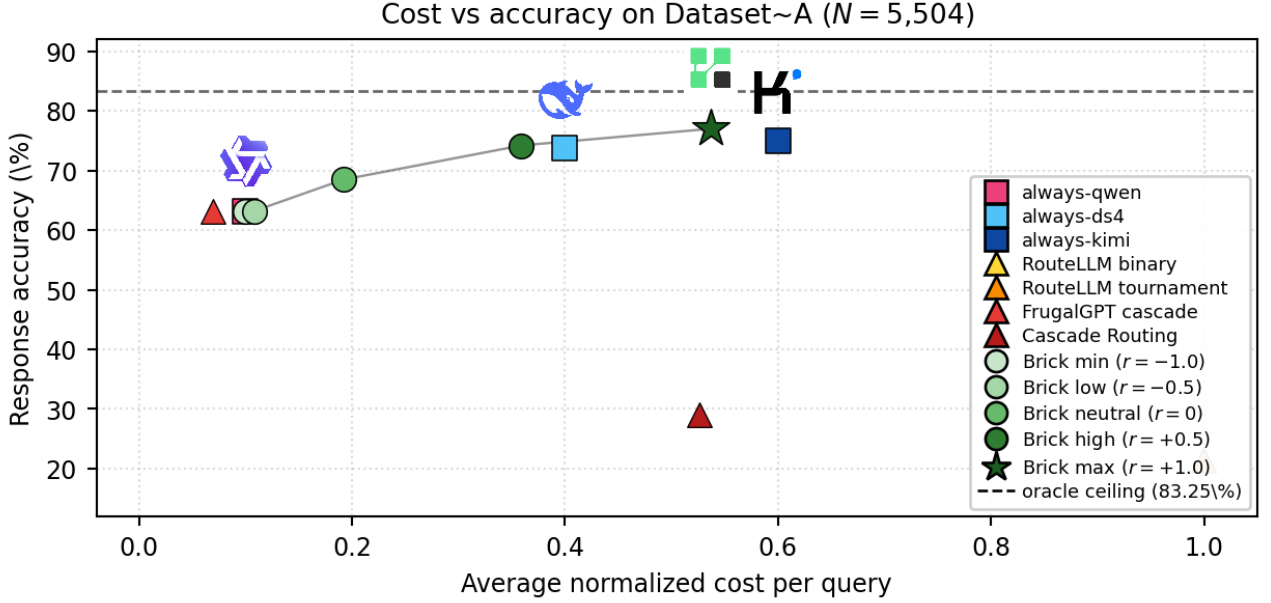


Figure 1. Cost vs response accuracy on Dataset A. Single-model baselines are squares, external routers are triangles, Brick (MoM) profiles are circles (max profile is a star). The dashed line marks the three-model oracle ceiling.

8. BRICK: A CALIBRATED MOM ROUTER

8.1. Pipeline

Brick processes each query through a deterministic six-step pipeline:

1. normalize and truncate the query text;
2. match the query against high-precision keyword rules (e.g. explicit code fences) that can bias the capability distribution;
3. compute a capability probability vector $p(x) \in \Delta^{D-1}$ (i.e. $p_c \geq 0$ and $\sum_c p_c = 1$) over a six-dimensional basis using a fine-tuned ModernBERT classifier;
4. compute a complexity label and confidence using a separate complexity classifier, mapping to a base difficulty level τ_q ;
5. compute a per-model capability score J_m using the routing math of §8.2;
6. select $m^* = \arg \min_m J_m$ with deterministic tie-breaking.

8.2. Routing math

Let $\mathcal{C} = \{c_1, \dots, c_D\}$ denote the capability basis with $D=6$ (coding, creative_synthesis, instruction_following, math_reasoning, planning_agentic, world_knowledge), and let $\mathcal{M} = \{m_1, \dots, m_K\}$ be the model pool of size $K=3$ (§4). Each model m has a non-negative cost scalar a_m (the dimensionless USD ratio of §4) and a vector of per-capability success rates $s_{m,c} \in (0, 1)$, collected in a skill matrix $S \in (0, 1)^{K \times D}$.

Skill matrix calibration. Each entry $s_{m,c}$ is a Bayesian-smoothed accuracy estimated offline on the results subset of Dataset A:

$$s_{m,c} = \frac{K_{m,c} + k \mu_c}{N_{m,c} + k}, \quad k = 8,$$

where $K_{m,c}$ counts the queries of capability c that model m answered correctly, $N_{m,c}$ is the total support for that (model, capability) cell, μ_c is the cross-model accuracy prior on capability c , and k is the prior strength (a Beta-Binomial pseudo-count that protects sparse cells from collapsing to 0 or 1). No target-model

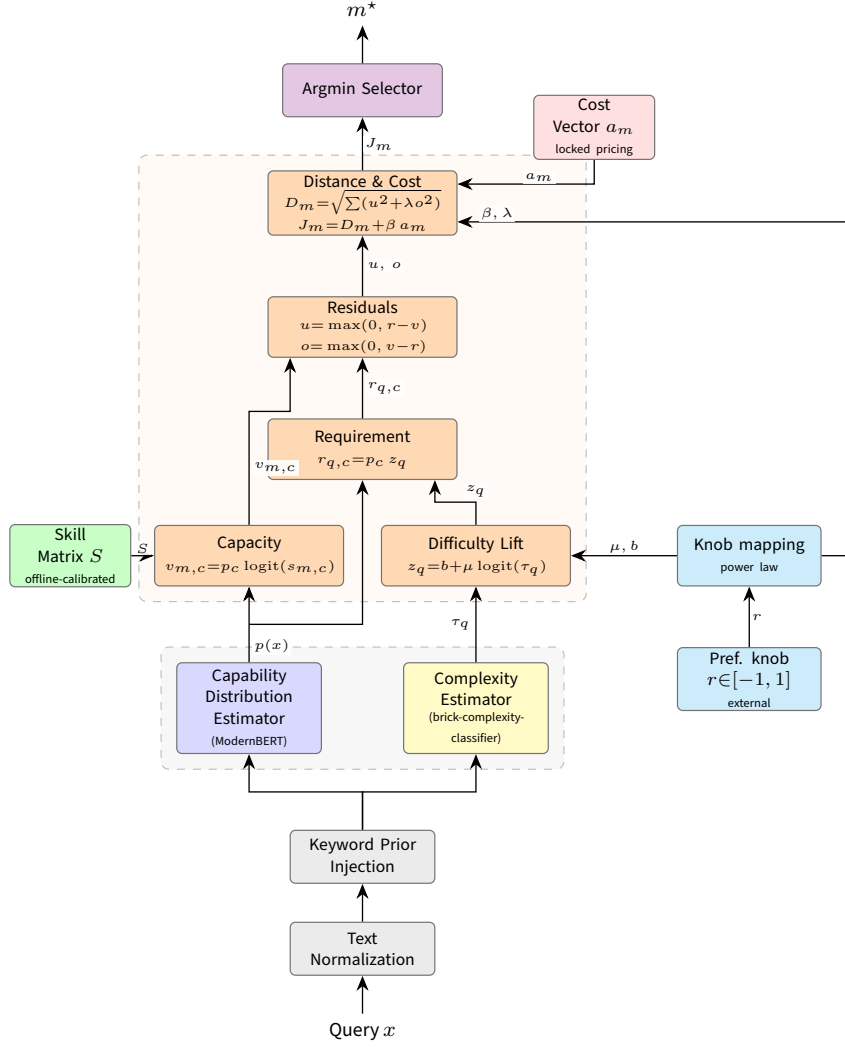


Figure 2. Brick architecture. The query x flows through text normalization and keyword prior injection, then branches into two parallel estimators: a ModernBERT capability distribution head producing $p(x) \in \Delta^{D-1}$, and a complexity head producing the difficulty target τ_q . The routing math is decomposed into five sub-blocks: difficulty lift (raising τ_q to logit space z_q), per-capability requirement $r_{q,c}$ and per-model capacity $v_{m,c}$ builders, asymmetric under/over-capacity residuals ($u_{m,c}, o_{m,c}$), and the cost-penalized distance J_m . Three side inputs feed the math: the offline-calibrated skill matrix $S \in (0, 1)^{K \times D}$ enters the capacity builder, the locked cost vector a_m enters the distance & cost combiner, and the external user-preference knob $r \in [-1, 1]$ is mapped via a calibrated power law to the four routing scalars (μ, b) feeding the difficulty lift and (β, λ) feeding the distance & cost block. The argmin selector returns m^* .

weights are updated; S is locked into the production configuration after calibration.

We chose Bayesian smoothing over plain frequentist accuracy because per-cell support $N_{m,c}$ is uneven across capabilities (some dimensions carry far fewer queries than others) and an unlucky cell of 0/5 correct answers would collapse $s_{m,c} \rightarrow 0$, sending $\logit(s_{m,c}) \rightarrow -\infty$ and producing a divergent per-model capacity downstream. The prior μ_c is the cross-model average accuracy on capability c rather than a flat 0.5 Laplace prior: sparse cells therefore fall back to the *typical* performance of the pool on that capability, not to a noninformative coin-flip that would unfairly advantage weak models on dimensions everybody struggles with. The pseudo-count $k=8$ is calibrated so that the prior dominates only when support is below roughly one tenth of the median per-cell sample size in Dataset A; above that, the data wins and

the estimate converges to the empirical mean.

Per-query scoring. For an incoming query, the ModernBERT classifier (§8.3) produces a capability distribution $p(x) = (p_{c_1}, \dots, p_{c_D})$ and the complexity head produces a blended difficulty $\tau_q \in (0, 1)$. We map difficulty to an *effective difficulty logit*

$$z_q = b + \mu \logit(\tau_q),$$

where b is an additive bias and μ a multiplicative slope (both set by the user-preference knob, §8.5). The choice of the logit transform is not cosmetic: a raw τ_q in probability space saturates near the boundaries and treats a step from 0.50 to 0.55 as equivalent to a step from 0.94 to 0.99, even though the second interval separates models that fail from oracle-level behaviour. Mapping difficulty to log-odds via

$\text{logit}(p) = \log(p/(1-p))$ removes that saturation and, crucially, places difficulty on the *same additive scale* as the per-model skill $\text{logit}(s_{m,c})$ in the capacity term below, so that the residual $r_{q,c} - v_{m,c} = p_c(z_q - \text{logit}(s_{m,c}))$ is interpretable as a per-capability *log-odds gap* rather than a raw probability difference. The affine form $b + \mu \text{logit}(\tau_q)$ is the minimum-sufficient parametrization on top of that: μ acts as a sensitivity (how much the difficulty signal matters; $\mu \rightarrow 0$ removes the dependence on the difficulty classifier, collapsing the requirement to $r_{q,c} = p_c b$, while the capacity comparison via $\text{logit}(s_{m,c})$ and the cost penalty βa_m remain active), b is a baseline (even easy queries demand a capability floor), and these are exactly the two scalar handles the preference knob r modulates in §8.5.

The distinction between τ_q and z_q is deliberate and not just a change of variable: τ_q is the *classifier's measurement* of difficulty (a probability the complexity head reports, with no operator input), while z_q is the *operator-calibrated difficulty* actually consumed by the math. The preference knob r must not touch τ_q (we do not falsify the classifier's own opinion); it acts one layer above, on (μ, b) , and the result is exposed under a separate name z_q . This single scalar then fans out across the six capabilities via p_c in the next step, so naming it explicitly avoids either inlining a long expression six times or, worse, attaching per-capability lifting parameters that would inflate the calibration surface from two scalars to $2D$.

The *per-capability requirement* is

$$r_{q,c} = p_c z_q,$$

read as “how hard the query is on capability c , weighted by how much that capability matters for this query”. The *per-model per-capability capacity* is

$$v_{m,c} = p_c \text{logit}(s_{m,c}),$$

the analogous quantity for the model. Both quantities carry the same factor p_c on purpose: it makes the gap $r_{q,c} - v_{m,c} = p_c(z_q - \text{logit}(s_{m,c}))$ proportional to how much capability c actually matters for this query, so a dimension with $p_c = 0$ contributes nothing to the distance no matter how badly any model fails on it. This is the structural difference between Brick and a domain-router: instead of picking one capability and discarding the rest, we form a continuous weighted comparison over all six dimensions and let the math discount the irrelevant ones automatically.

We then decompose the gap into two asymmetric residuals:

$$u_{m,c} = \max(0, r_{q,c} - v_{m,c}),$$

$$o_{m,c} = \max(0, v_{m,c} - r_{q,c}),$$

the *under-capacity* u (model is too weak on c) and the *over-capacity* o (model is overkill on c). We split the residual into two non-negative halves instead of penalizing the symmetric $(r_{q,c} - v_{m,c})^2$ because the two failure modes carry asymmetric consequences. Under-capacity translates directly into a higher chance of

a wrong answer (the model lacks the skill the query asks for); over-capacity is mostly wasted money and latency (the model could have been smaller). We want the routing score to weight these two regimes independently, which the symmetric absolute residual cannot express. They combine in a weighted Euclidean capability distance and a cost-penalized score:

$$D_m = \sqrt{\sum_c (u_{m,c}^2 + \lambda o_{m,c}^2)}, \quad J_m = D_m + \beta a_m,$$

where D_m is the *capability distance* (a pure quality term: the log-odds gap between what the query needs and what model m offers, aggregated over the six capabilities), J_m is the *routing objective* (the cost function the router actually minimizes, in the optimization-theory sense of an objective to be driven to its argmin), $\lambda \geq 0$ is the *over-capacity penalty* (smaller λ tolerates overkill, recovering the standard quality-only objective in the limit $\lambda \rightarrow 0$), and $\beta \geq 0$ is the *cost coefficient* (larger β trades quality for price). The router selects $m^* = \arg \min_m J_m$, i.e. the model whose quality gap plus cost penalty is smallest; with $\beta = 0$ this collapses to argmin of D_m alone (quality-only oracle), and with $\beta \rightarrow \infty$ it collapses to argmin of a_m (cheapest model regardless of quality). In production, ties within a calibrated band $|J_i - J_j| < \varepsilon_\tau$ (default $\varepsilon_\tau = 0.03$) are broken deterministically by Expected-Success $\sum_c p_c s_{m,c}$ (higher is preferred), then by lower cost a_m ; the arg min rule above is the unbanded idealized form. See `brickrouting/router.go` for the exact implementation.

A few more design choices are worth unpacking.

Why Euclidean aggregation. The aggregation across capabilities is Euclidean (L_2) rather than L_1 (sum of absolute gaps) or L_∞ (worst gap). L_1 would treat every per-capability gap as interchangeable currency and ignore how the deficit is distributed across dimensions; L_∞ would discard every capability except the single worst one. L_2 keeps all six dimensions in play and penalizes large residuals super-linearly, so a single sharp under-capacity gap weighs more than the same total mass split across several small ones, which matches the empirical observation that a model failing badly on the dominant capability of a query is harder to recover than one that is mildly off across the board.

Why λ multiplies only o^2 . The asymmetric weight λ rescales only the over-capacity branch, so the two regimes can be tuned independently: pushing $\lambda \rightarrow 0$ recovers the standard distance-to-floor objective in which overkill is free, while increasing λ progressively punishes wasted capability (typically a proxy for cost or latency we cannot price directly).

Why the cost penalty is additive. The term βa_m is added linearly to D_m rather than multiplied because the two quantities live in incommensurable units (a

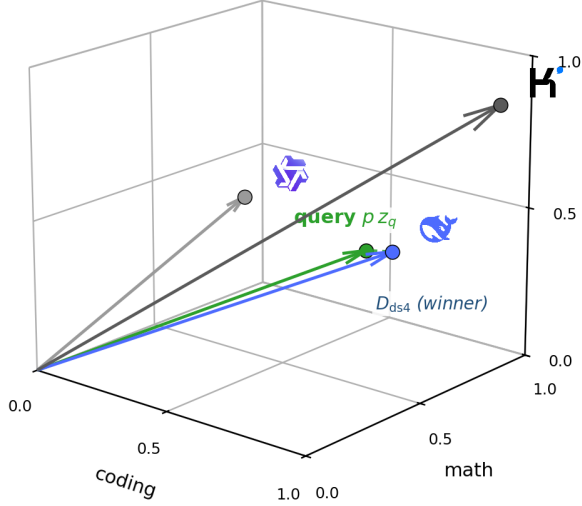


Figure 3. Illustrative projection onto three of the six capability axes. The Brick router operates in the full six-dimensional capability basis; this 3D rendering is schematic. The dashed arc marks the winning distance to the selected model.

log-odds gap and a dimensionless dollar ratio), and a multiplicative form would force them to share scale before any trade-off has even been defined. The additive form treats β as the explicit conversion rate from cost units into distance units, so operators can read off how much extra log-odds gap they are willing to accept per dollar of saving.

Why these four scalars are not hyperparameters. The four scalars (μ, b, β, λ) are not free knobs sitting in a config file but functions of the preference knob $r \in [-1, 1]$ exposed at routing time (§8.2).

Figure 3 sketches the geometry on three of the six capability axes.

8.3. ModernBERT capability classifier

Step (iii) is a fine-tuned ModernBERT classifier [16] that maps a query string to a probability vector over six capability dimensions. We fine-tune `answerdotai/ModernBERT-base` on Dataset B, a 50k-query labeled corpus distinct from Dataset A. Training is a W&B sweep over learning rate (10^{-5} to 10^{-4} , log), weight decay (10^{-6} to 10^{-2}), warmup ratio (0.03 to 0.12), and number of epochs (2 to 5). The winning configuration is selected by the macro-Pearson correlation on a held-out evaluation split, reaching 0.983.

- [HuggingFace: regolo/brick-modernbert-capability-classifier](#)
- [W&B sweep: dataset-b-modernbert / 0srgzjrg](#)

8.4. Complexity classifier and difficulty blending

Step (iv) of the pipeline is a separate fine-tuned classifier that scores how hard the query is for the pool.

We use a 3-class head (easy/medium/hard) built on Qwen3.5-0.8B with LoRA adapters ($r=\alpha=32$, dropout 0.1), fine-tuned for 3 epochs at learning rate 10^{-4} , batch size 16, maximum sequence length 768. Training uses an asymmetric over-penalty ($\lambda_{\text{over}}=0.7$, biasing the classifier to under- rather than over-rate difficulty) and label smoothing 0.08; on the held-out test set ($n=1994$) it reaches 72.8% accuracy and 0.425 macro-F1. The classifier returns a label $\ell \in \{\text{easy}, \text{medium}, \text{hard}\}$ and a confidence $c \in [0, 1]$.

Three anchor values bind labels to a continuous difficulty axis,

$$\tau_{\text{easy}}=0.55, \quad \tau_{\text{medium}}=0.72, \quad \tau_{\text{hard}}=0.88,$$

and the blended difficulty τ_q consumed by the router (§8.2) is a confidence-weighted convex combination of the predicted-label anchor and the medium anchor:

$$\tau_q = c \cdot \tau_\ell + (1 - c) \cdot \tau_{\text{medium}}.$$

When the classifier is uncertain ($c \rightarrow 0$) the difficulty collapses to the safe medium tier ($\tau_q \rightarrow \tau_{\text{medium}}$); when it is confident ($c \rightarrow 1$) it matches the predicted anchor. This blending replaces a brittle hard decision with a smooth contraction toward the centre, absorbing the residual miscalibration of the small classifier without escalating to a larger model just to break a tie. As a concrete example, the query of §8.6 is labelled hard with $c=0.51$, yielding $\tau_q = 0.51 \cdot 0.88 + 0.49 \cdot 0.72 = 0.802$.

- [HuggingFace: regolo/brick-complexity-2-eco](#)

8.5. User preference knob

Brick exposes a continuous preference $r \in [-1, 1]$ with named profiles `min` $\equiv -1$, `neutral` $\equiv 0$, `max` $\equiv +1$. The knob smoothly modulates the four routing scalars ($\mu(r), b(r), \beta(r), \lambda(r)$) that enter the scoring of §8.2. Concretely, define the positive and negative branches

$$p^+(r) = \max(r, 0)^\alpha, \quad p^-(r) = \max(-r, 0)^\alpha,$$

where $\alpha > 0$ is a sharpness exponent (larger α flattens the response near $r=0$ and concentrates the action at the extremes). The four scalars follow a calibrated asymmetric power law:

$$\begin{aligned} \mu(r) &= \mu_0 \exp(p^+ \ln A_\mu^+ + p^- \ln A_\mu^-), \\ b(r) &= b_0 + p^+ A_b^+ + p^- A_b^-, \\ \beta(r) &= \beta_0 \exp(-p^+ \ln A_\beta^+ + p^- \ln A_\beta^-), \\ \lambda(r) &= \lambda_0 \exp(-p^+ \ln A_\lambda^+ + p^- \ln A_\lambda^-). \end{aligned}$$

The four base values ($\mu_0, b_0, \beta_0, \lambda_0$) are the neutral-knob ($r=0$) defaults; the eight $A^\pm$ are asymmetric multipliers that let calibration push harder on one side of the knob than the other. The construction is not arbitrary; below we trace where it comes from, then justify each design choice in turn.

Where these formulas come from. Three constraints shape the family: continuity in r , identity

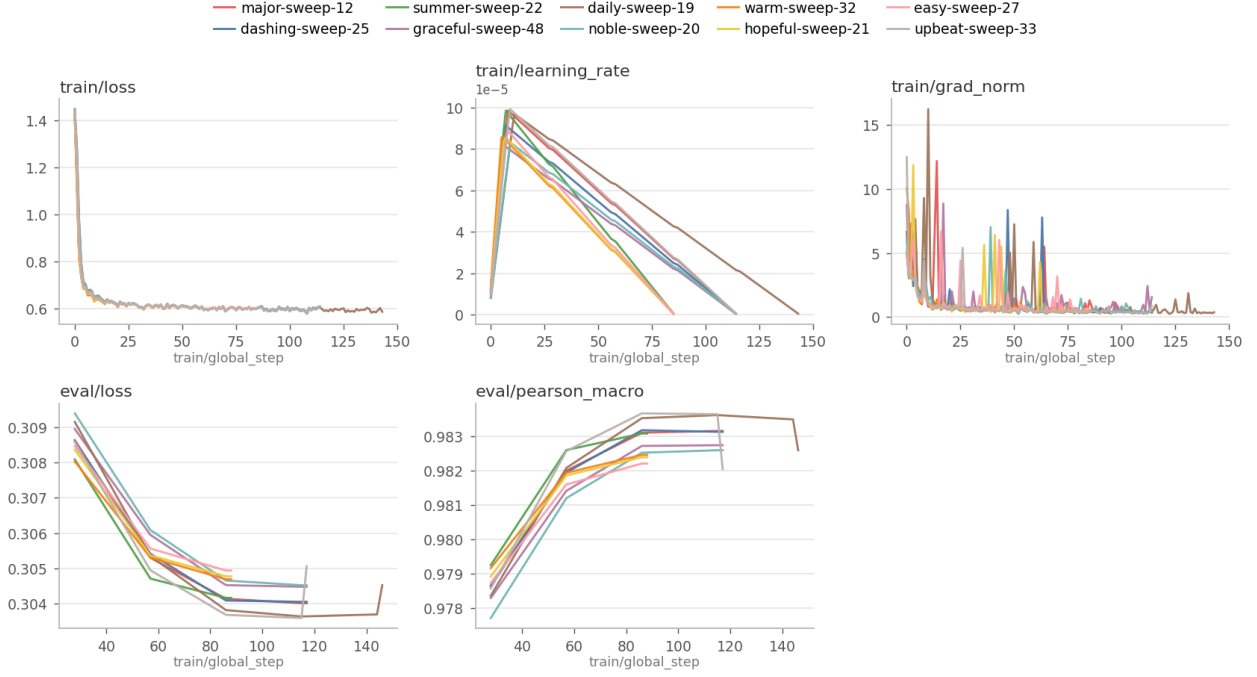


Figure 4. Top-10 runs from the ModernBERT W&B sweep by Pearson-macro, in a multi-metric dashboard view (training loss, learning rate, gradient norm, validation loss, validation Pearson macro).

at $r=0$ (neutral reproduces production defaults), and pre-calibrated targets at $r = \pm 1$. These constraints are met by many continuous families; we pick a one-sided power law $|r|^\alpha$ as the simplest interpretable parametrization that allows independent asymmetric calibration of the two extremes, hence $|r|^\alpha$ as the building block. Since the two extremes are not mirror images (e.g. λ must shrink by $\approx 900\times$ at $r=+1$ but inflate only $\approx 12\times$ at $r=-1$), we split into one-sided basis functions $p^+(r) = \max(r, 0)^\alpha$ and $p^-(r) = \max(-r, 0)^\alpha$: both vanish at 0, exactly one is unity at ± 1 , never overlap. The combiner is then dictated by domain: μ, β, λ must stay positive, so $x_0 \cdot \exp(\cdot)$; b lives in logit space, so $b_0 + (\cdot)$. We wrap the calibration constants in $\ln$ inside the exponent purely for readability: at $r = \pm 1$ the exponent collapses to $\pm \ln A$, so $A^\pm$ is literally the multiplicative factor at each extreme (e.g. $A_\lambda^+ = 896$ reads off the table as “ λ shrinks $896\times$ at max-quality”). The sign in front of each $p^\pm \ln A^\pm$ is fixed by routing semantics: at $r=+1$ we want β, λ to shrink (minus sign) and μ to grow (plus sign); b stays straight additive.

Why two one-sided branches p^+, p^- instead of a signed $|r|^\alpha$. A signed term would still vanish at $r=0$ but would force a single constant to govern both extremes. We wanted independent calibration per side because the production trade-offs are not symmetric: for example, the over-capacity penalty λ has to deflate by a factor of about 900 at $r=+1$ (so overkill stops mattering when the operator picks max-quality) but only needs to inflate by a factor of about 12 at $r=-1$ (more than enough to push the router toward cheap models). That ratio of ratios cannot be expressed with a single multiplier; the split into p^+, p^- with independent

A^+, A^- is the cheapest parametrization that buys per-side freedom while keeping continuity and intrinsic neutrality at $r=0$.

Why log-multiplicative form for μ, β, λ . These three scalars are *magnitudes* that must stay strictly positive: μ scales a logit, β scales an additive cost, λ scales a sum of squares. Writing each one as $x_0 \cdot \exp(\dots)$ enforces positivity for free regardless of what calibration puts inside the exponent. Putting $\pm p^\pm \ln A$ inside, then, gives exactly $x_0 \cdot A^{\mp 1}$ at $r = \pm 1$: the constants A^+ and A^- can then be read directly off the table as the multiplicative factors (or their inverses, depending on the sign convention) by which each scalar changes at the corresponding extreme. The calibration is interpretable by inspection.

Why additive form for b . The bias b enters the model as $z_q = b + \mu \text{logit}(\tau_q)$, i.e. it has the dimension of a logit (real-valued, sign-bearing). Forcing positivity through $\exp$ would be wrong here; the natural choice is an additive shift, and the constants $A_b^\pm$ then express *how many units of log-odds* the baseline moves at each extreme. For instance, $A_b^- = -5.87$ subtracts almost six log-odds from b at $r=-1$, collapsing the requirement vector and aggressively cheapening the dispatch.

Why the signs on p^+, p^- are inverted for β, λ . By convention $r=+1$ is the max-quality profile: in that regime we want the cost coefficient β and the over-capacity penalty λ to be *small* so the router stops worrying about price and about overkill. The negative sign in front of $p^+ \ln A_\beta^+$ produces β_0/A_β^+ at $r=+1$

(with $A_\beta^+=13.4$ that is $\beta \approx 0.63/13.4 \approx 0.047$: cost becomes nearly free, the router chases quality); the positive sign in front of $p^- \ln A_\beta^-$ produces $\beta_0 \cdot A_\beta^-$ at $r=-1$ (with $A_\beta^-=13$ that is $\beta \approx 0.63 \cdot 13 \approx 8.2$: cost dominates, the router collapses to the cheapest model). The same construction holds for λ . For μ the sign convention is the opposite: $r=+1$ should *inflate* the difficulty signal so larger models qualify, hence the $+p^+ \ln A_\mu^+$ in the exponent. The b branch is simpler: $r=+1$ pushes b upward by A_b^+ (queries look harder) and $r=-1$ pulls b downward by A_b^- (queries look easier).

The locked production values are listed in Table 4.

At inference, the knob is passed per-request via the HTTP header X-Brick-Routing-Preference (a float in $[-1,1]$) or the convenience header X-Brick-Routing-Profile with named values eco/balanced/pro mapping to $r=-1, 0, +1$. The thirteen power-law constants are loaded once from the service configuration at startup and never recomputed at routing time.

8.6. Worked example

Query. q_03563 (dim: planning_agentic): “Can you help me with a few tasks? First, I am playing a game where I need to calculate the evolutionary fitness of a creature. ...”

Step 0 – preference knob. The caller sets $r=0$ (balanced profile), so $p^+=p^-=0$ and the four routing scalars collapse to their locked production base values (Table 4):

$$(\mu, b, \beta, \lambda) = (\mu_0, b_0, \beta_0, \lambda_0) = (1.07, 0.15, 0.63, 0.35).$$

Step 1 – capability distribution $p(x)$ from ModernBERT (six entries, sum to one):

c	cod	crea	ifo	math	plan	wld
p_c	0.09	0.53	0.09	0.09	0.09	0.09

(Values rounded to two decimals for display; full-precision probabilities sum to one.)

Step 2 – complexity and difficulty lift. The complexity head labels the query hard with confidence 0.51, blending into $\tau_q=0.802$. The effective difficulty logit is

$$z_q = b + \mu \logit(\tau_q) = 0.15 + 1.07 \cdot 1.401 = 1.649.$$

Step 3 – per-capability requirement $r_{q,c}=p_c z_q$ (with $z_q=1.649$):

c	cod	crea	ifo	math	plan	wld
p_c	0.09	0.53	0.09	0.09	0.09	0.09
$r_{q,c}$	0.16	0.87	0.16	0.16	0.16	0.16

Step 4 – per-model capacity $v_{m,c}=p_c \logit(s_{m,c})$, with S the offline-calibrated skill matrix:

c	cod	crea	ifo	math	plan	wld
$s_{qwen,c}$	0.715	0.512	0.810	0.912	0.577	0.180
$v_{qwen,c}$	0.09	0.02	0.14	0.22	0.03	-0.14
$s_{ds4,c}$	0.821	0.658	0.863	0.935	0.621	0.489
$v_{ds4,c}$	0.14	0.34	0.17	0.25	0.05	-0.00
$s_{kimi,c}$	0.904	0.752	0.870	0.944	0.642	0.344
$v_{kimi,c}$	0.21	0.58	0.18	0.27	0.06	-0.06

Step 5 – asymmetric residuals.

$u_{m,c} = \max(0, r_{q,c} - v_{m,c})$ (under-capacity, model too weak), $o_{m,c} = \max(0, v_{m,c} - r_{q,c})$ (over-capacity, overkill):

c	cod	crea	ifo	math	plan	wld
$u_{qwen,c}$	0.07	0.84	0.02	0.00	0.13	0.30
$o_{qwen,c}$	0.00	0.00	0.00	0.07	0.00	0.00
$u_{ds4,c}$	0.01	0.52	0.00	0.00	0.11	0.16
$o_{ds4,c}$	0.00	0.00	0.02	0.10	0.00	0.00
$u_{kimi,c}$	0.00	0.29	0.00	0.00	0.10	0.22
$o_{kimi,c}$	0.06	0.00	0.02	0.11	0.00	0.00

Step 6 – distance, cost penalty, total score.

$D_m = \sqrt{\sum_c (u_{m,c}^2 + \lambda o_{m,c}^2)}$ (with $\lambda=0.35$) and $J_m = D_m + \beta a_m$ (with $\beta=0.63$):

model	a_m	D_m	βa_m	J_m	sel.
 qwen	0.10	0.908	0.063	0.971	
 ds4	0.40	0.562	0.252	0.814	
 kimi	0.60	0.380	0.378	0.758	✓

Step 7 – decision. $m^* = \arg \min_m J_m = \mathbf{K}^{\text{kimi}2.6}$ ($J_{\text{qwen}}=0.971$, $J_{\text{ds4}}=0.814$, $J_{\text{kimi}}=0.758$). The dominant capability is creative_synthesis at $p=0.53$, and $\tau_q=0.80$ inflates the per-capability requirement; the qwen capacity on that dimension falls short, so its under-capacity penalty exceeds the cost gap to kimi.

Table 5. Two additional Brick decisions across capability dimensions, with the dominant capability probability and complexity-derived difficulty.

Dimension	Dominant cap.	τ_q	Selected
coding	crea=0.66	0.56	 qwen
planning_agentic	pln=0.57	0.72	 kimi

9. BRICK RESULTS ON DATASET A

Table 6 reports Brick’s full-dataset out-of-fold curve under stratified 5-fold skill-vector calibration. For each fold, skill vectors are calibrated on the four held-in folds and evaluated on the held-out fold; knob hyperparameters are fixed from prior development. This protocol uses the entire dataset while avoiding per-example leakage in the skill estimates. It should be read as an in-domain calibrated evaluation, not as a zero-shot result.

Table 4. Locked production math configuration for the Brick preference knob (symbols above, values below).

μ_0	b_0	β_0	λ_0	α	A_μ^+	A_b^+	A_β^+	A_λ^+	A_μ^-	A_b^-	A_β^-	A_λ^-
1.07	0.15	0.63	0.35	1.56	3.24	1.05	13.4	896	0.377	-5.87	13	11.9

Table 6. Brick out-of-fold results on Dataset A under locked knob hyperparameters.

Profile	r	Sel. acc.	Avg. cost	Dist. (%)
min	-1.0	63.17%	0.100	100/0/0
low	-0.5	63.17%	0.100	100/0/0
neutral	0.0	68.46%	0.192	71/25/3
high	+0.5	74.18%	0.359	25/59/16
max	+1.0	76.98%	0.537	0/31/69

Skill estimator note. The skill matrix S described in §8.2 is the production-locked, Bayesian-smoothed estimator with pseudo-count $k=8$. The per-fold skill vectors used to produce Table 6 are computed by a different routine: a probability-weighted empirical accuracy

$$s_{m,c} = \text{clip}\left(\frac{\sum_q p_c(q) \mathbb{I}[m \text{ correct on } q]}{\sum_q p_c(q)}, 0.02, 0.98\right),$$

with soft capability weights $p_c(q)$ from the ModernBERT classifier and a hard floor/ceiling at $[0.02, 0.98]$ in lieu of an explicit prior. The two estimators converge in the large-support limit and the choice is empirically inconsequential for the routing decision at the per-fold scale of Dataset A.

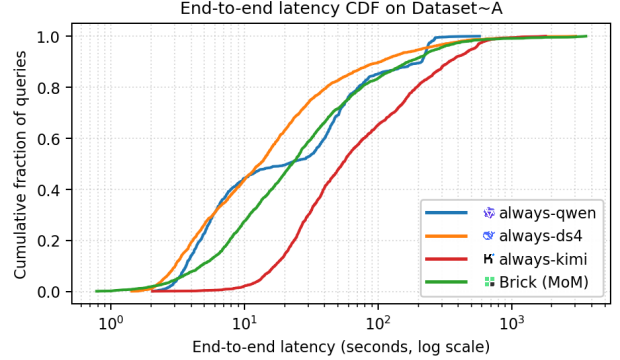
The max-quality profile reaches 76.98%, exceeding always-kimi (75.02%) by +1.96 pp while spending 0.537 vs 0.600 in normalized cost. The neutral profile sits at 68.46% (+5.29 pp over always-qwen) at cost 0.192, a roughly $5\times$ cheaper operating point than always-kimi. The gap to the 83.25% oracle is the residual routing opportunity not captured by the current six-dimensional capability features and the deterministic decision rule.

10. LATENCY ANALYSIS

We report latency along two axes: the raw *router overhead* (decision time, LLM call excluded) and the *perceived end-to-end latency* (router decision plus LLM generation time).

Table 7. Router decision overhead (LLM call excluded) on Dataset A ($N=5,504$). All values in milliseconds.

Router	p50	p95	p99	mean
Brick (ours)	2998	3211	3311	1883
Cascade Routing	15.1	19.7	48.6	16.0
FrugalGPT	103.3	157.7	556.9	90.2
RouteLLM binary	97.5	138.3	394.6	84.0
RouteLLM tournament	197.8	306.1	1318	175.2

**Figure 5.** End-to-end latency CDF on Dataset A. Brick (MoM) at the max profile sits between always-ds4 and always-kimi, while achieving higher response accuracy than either.**Table 8.** Perceived end-to-end latency (router decision plus LLM generation) on Dataset A. The MoM (Brick max profile) row joins Brick’s router latency with the LLM latency of the selected model per query. Single-model baselines show LLM generation latency only.

System	p50	p95	p99	mean
always-qwen	21441	232232	266774	52677
always-ds4	10169	135296	468067	41768
always-kimi	51246	390276	738106	106926
Brick (MoM)	22802	278621	820288	78178

The router-overhead numbers in Table 7 reflect the Dataset A benchmark harness, where Brick makes two auxiliary classifier calls (capability and complexity) over a remote endpoint, while Cascade Routing performs a local hash lookup. In a production deployment of Brick observed over 6,376 live requests, the same decision step has a median of 494 ms (p95 2,528 ms), reflecting warm caches and co-located classifiers. The benchmark overhead is therefore a conservative upper bound on practical decision latency.

For perceived latency (Table 8), Brick at the max-quality profile reaches the same response-accuracy regime as always-kimi at less than half its median end-to-end latency (22.8 s vs 51.2 s). The reason is that even at the max profile Brick routes roughly 31% of queries to deepseek-v4-flash, which is materially faster than kimi2.6 per generation. Figure 5 shows the latency CDFs: Brick’s distribution sits to the left of always-kimi at every quantile, and rejoins always-ds4 at the median.

11. DISCUSSION AND LIMITATIONS

The reported results support three claims. (i) MoM dispatches queries across whole models and is a vi-

able design point distinct from MoE. (ii) Within a three-model pool, a calibrated capability-aware router (Brick) exceeds the best single-model baseline on response accuracy at lower cost and lower perceived latency. (iii) The headroom above the best router and below the oracle ($83.25\% - 76.98\% = 6.27$ pp) reflects information the router does not yet extract from the query and the model skill matrix; this is the direction where future MoM routers will compete.

The current evaluation is in-domain calibrated: the skill matrix is fit on held-in folds of Dataset A, and the knob hyperparameters were selected on Dataset A. Strict external generalization should be measured on a fully out-of-distribution benchmark before claims are extended to arbitrary deployment workloads. The reported latency numbers assume a public-API deployment for the LLM call and a remote classifier endpoint for the router; numbers will move under self-hosted inference.

The MoM paradigm naturally extends to mixed open-weight and closed-weight pools. The router treats commercial APIs as additional points in the cost-capability plane. The principal open work is calibrating skill vectors for closed-weight models without leaking proprietary evaluation data and propagating their non-stationary pricing into the cost vector a .

12. CONCLUSION

We argued that whole-model routing across heterogeneous LLM pools, framed as *Mixture of Models*, is a deployment pattern worth treating as a first-class object. We instantiated it with Brick, a calibrated capability-aware router that exceeds the best single-model baseline and external routing baselines on Dataset A, at competitive latency. We see MoM as a practical bridge for operators who want to mix open-weight and commercial closed-weight models in a single product, paying for capacity only where the query demands it.

REFERENCES

- [1] D. Hendrycks et al., “Measuring Mathematical Problem Solving with the MATH Dataset,” in *Proc. NeurIPS Datasets and Benchmarks*, 2021.
- [2] K. Cobbe et al., “Training Verifiers to Solve Math Word Problems,” arXiv:2110.14168, 2021.
- [3] Y. Wang et al., “MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark,” in *Proc. NeurIPS Datasets and Benchmarks*, 2024.
- [4] J. Zhou et al., “Instruction-Following Evaluation for Large Language Models,” arXiv:2311.07911, 2023.
- [5] V. Pyatkin et al., “IFBench: Granular Instruction-Following Evaluation,” arXiv:2503.07879, 2025.
- [6] F. Yan et al., “Berkeley Function-Calling Leaderboard,” Berkeley AI Research, 2024.
- [7] N. Jain et al., “LiveCodeBench: Holistic and Contamination-Free Evaluation of Large Language Models for Code,” arXiv:2403.07974, 2024.
- [8] J. Wei et al., “SimpleQA: Measuring Short-Form Factuality in Large Language Models,” arXiv:2411.04368, 2024.
- [9] D. Rein et al., “GPQA: A Graduate-Level Google-Proof Q&A Benchmark,” in *Proc. COLM*, 2024.
- [10] S. Paech et al., “EQ-Bench-Creative-v3: Emotional Intelligence and Creative Writing Evaluation,” 2024.
- [11] Stanford NLP, “LitBench: A Literary Synthesis Benchmark for Long-Form Evaluation,” 2025.
- [12] S. Yao et al., “tau-bench: A Benchmark for Tool-Augmented Reasoning,” arXiv:2406.12045, 2024.
- [13] I. Ong et al., “RouteLLM: Learning to Route LLMs with Preference Data,” arXiv:2406.18665, 2024.
- [14] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance,” arXiv:2305.05176, 2023.
- [15] W. Jitkrittum et al., “Cascade Routing for Large Language Models,” arXiv:2405.20828, 2024.
- [16] B. Warner et al., “Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory-Efficient, and Long-Context Fine-Tuning and Inference,” arXiv:2412.13663, 2024.